

Listas Encadeadas - Aplicação: Ordenação Topológica

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Ordenação Topológica

- **Ordenação topológica** é um problema que pode ser caracterizado como uma aplicação de **listas encadeadas**
- **Importante**
 - Uso potencial todas as vezes em que o problema abordado envolve uma ordem parcial

Ordem Parcial e Ordenação Topológica

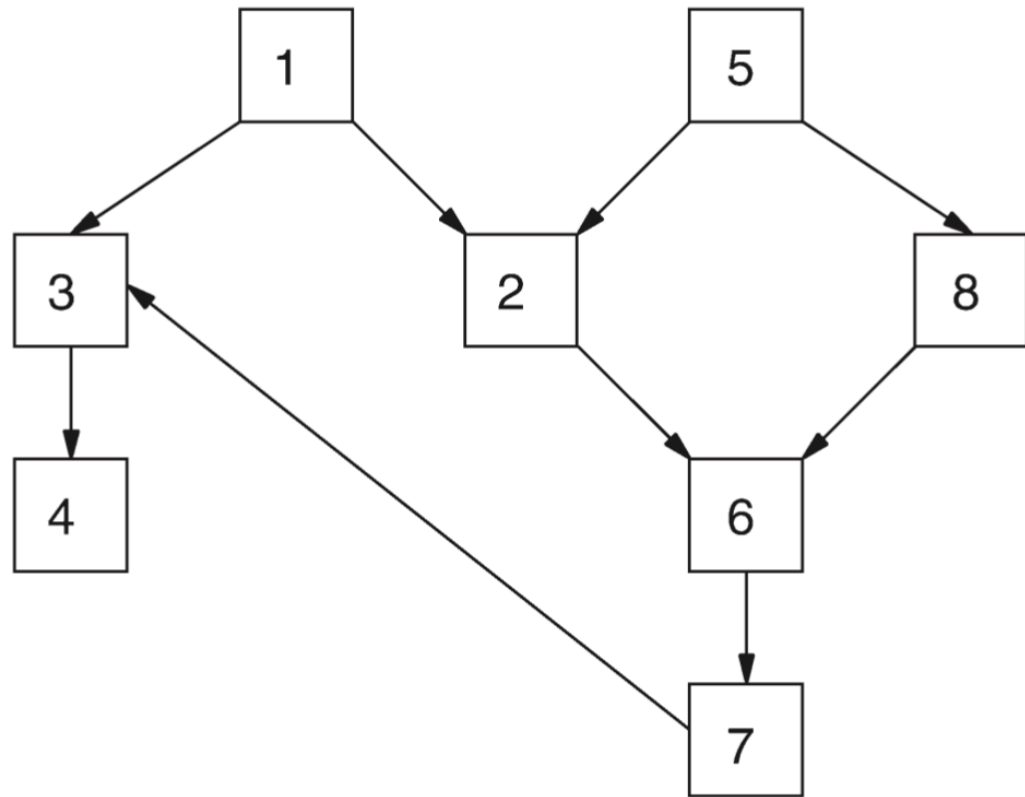


FIGURA 2.13 Representação de ordem parcial.

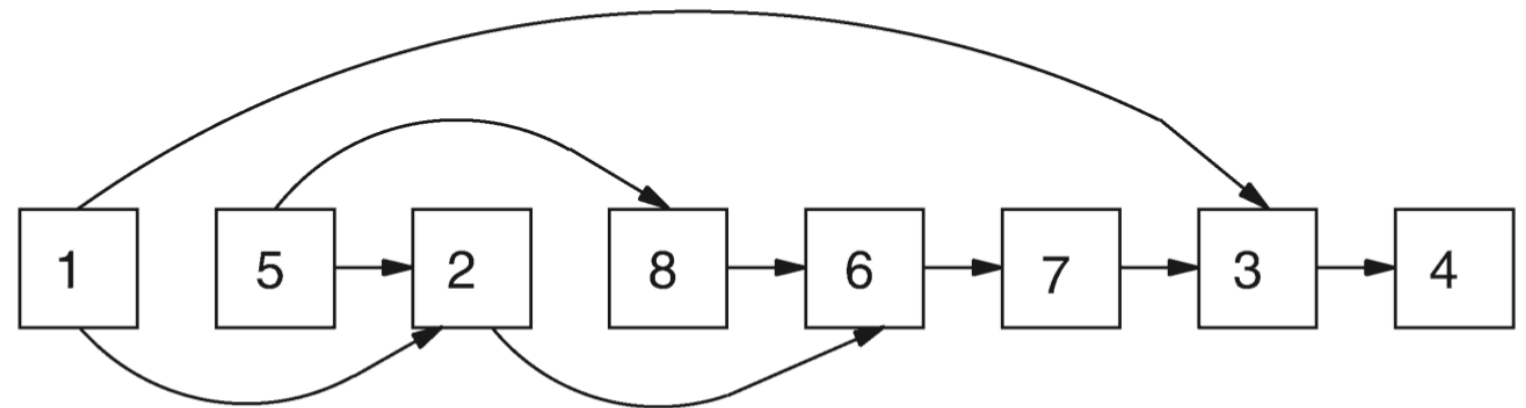


FIGURA 2.14 Ordenação topológica.

Ordem Parcial

- **Ordem parcial** de conjunto S é uma relação entre objetos de S , representada pelo símbolo " $\leq$ " (precede ou iguala)
 - $x \leq y$ pode ser lida " x precede ou iguala y "
- Satisfaz as seguintes propriedades para quaisquer objetos x , y e z , não necessariamente distintos em S :
 - (i) se $x \leq y$ e $y \leq z$, então $x \leq z$ (transitiva);
 - (ii) se $x \leq y$ e $y \leq x$, então $x = y$ (anti-simétrica);
 - (iii) $x \leq x$ (reflexiva).

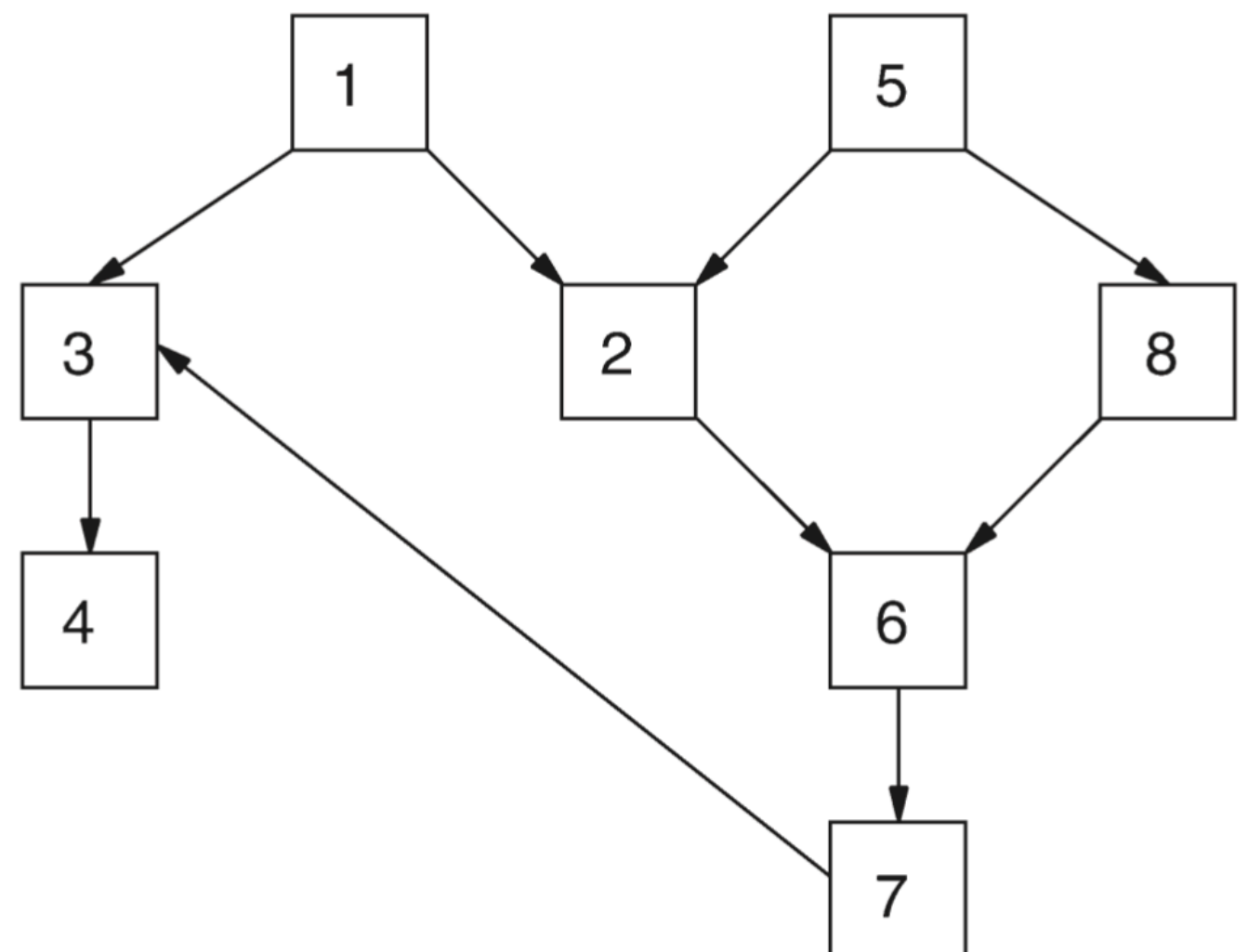


FIGURA 2.13 Representação de ordem parcial.

$x < y$ (x precede y)

- Se $x \leq y$ e $x \neq y$, se escreve $x < y$ e diz-se “ x precede y ”.
Tem-se então:
 - (i') se $x < y$ e $y < z$ então $x < z$ (transitiva);
 - (ii') se $x < y$, então $y \not< x$ (assimétrica);
 - (iii') $x \not< x$ (irreflexiva).
- Assume-se que S é um conjunto finito, uma vez que se deseja trabalhar no computador

Exemplo Ordem Parcial

- Muitos exemplos interessantes podem ser mencionados como utilização da ordem parcial!!
- Execução de um conjunto de tarefas necessárias, por exemplo, à montagem de um automóvel
 - Cada caixa uma tarefa
 - Cada tarefa numerada arbitrariamente
 - Se existe a indicação de um caminho da caixa x para a caixa y , isso significa que a tarefa x deve ser executada antes da tarefa y

Ordenação Topológica

- **Ordenação topológica** trata de obter uma **ordem linear** a partir de uma **ordem parcial**
 - Rearrumar os objetos numa sequência $a_1, a_2, \dots, a_n$ tal que sempre que $a_j \prec a_k$, tem-se $j < k$

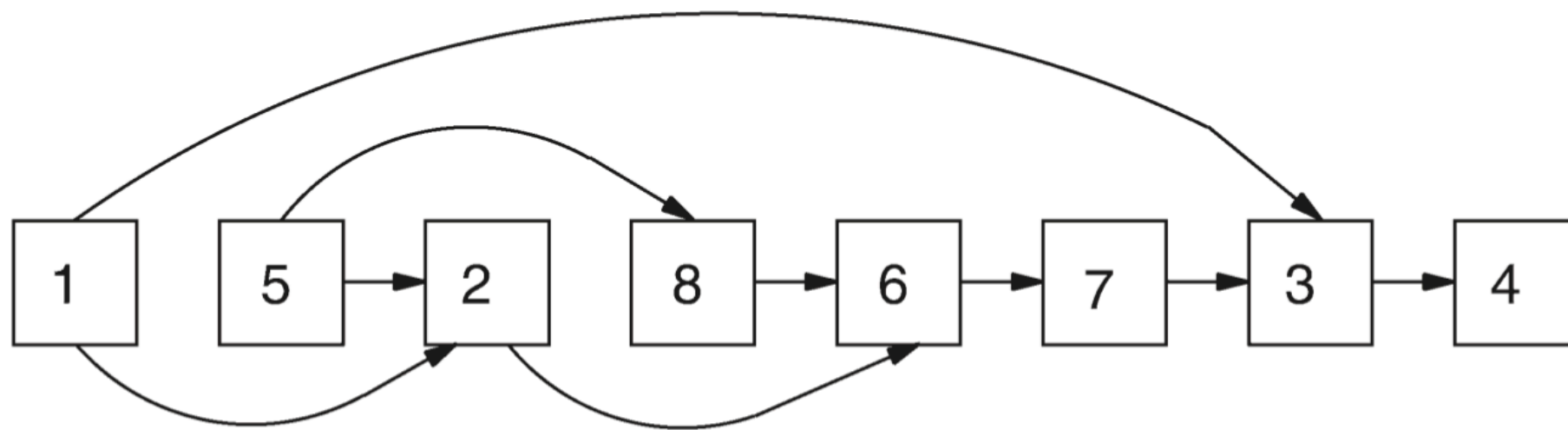


FIGURA 2.14 Ordenação topológica.

Ideia Ordenação Topológica

- Inicialmente, se considera um objeto que não é precedido por nenhum outro na ordem parcial
- Esse objeto é o primeiro na saída, isto é, na **ordem final**
- Agora, remova o objeto do conjunto S
- Conjunto resultante obedece novamente a uma **ordem parcial**
- Repetir até que todo o conjunto esteja ordenado

Ideia Ordenação Topológica (Cont.)

- Esse algoritmo só falharia se, em algum momento, a ordem parcial de um conjunto fosse tal que todos os elementos tivessem um predecessor
- Ora, isso é impossível, porque contraria as propriedades *(i)* e *(ii)*
 - Como o conjunto S é finito, assumir que todo elemento possui um predecessor implica construir uma cadeia infinita de predecessores, o que só é possível se algum elemento se repetir. Mas a repetição de um elemento na cadeia gera um ciclo. A existência de um ciclo, contudo, viola a anti-simetria da relação ($x \preceq y$ e $y \preceq x$ implica $x = y$) e, portanto, contradiz a definição de ordem parcial. Logo, em toda ordem parcial finita deve existir ao menos um elemento sem predecessor.

Implementação

- Desempenho desse algoritmo depende de sua implementação
- Objetos a serem ordenados são numerados de 1 a n , em qualquer ordem, e alocados sequencialmente na memória
- Entrada do algoritmo são os pares (j, k) , significando que o objeto j precede o objeto k
- Número de objetos n e o número de pares m também são fornecidos

Armazenamento para Ordenação Topológica

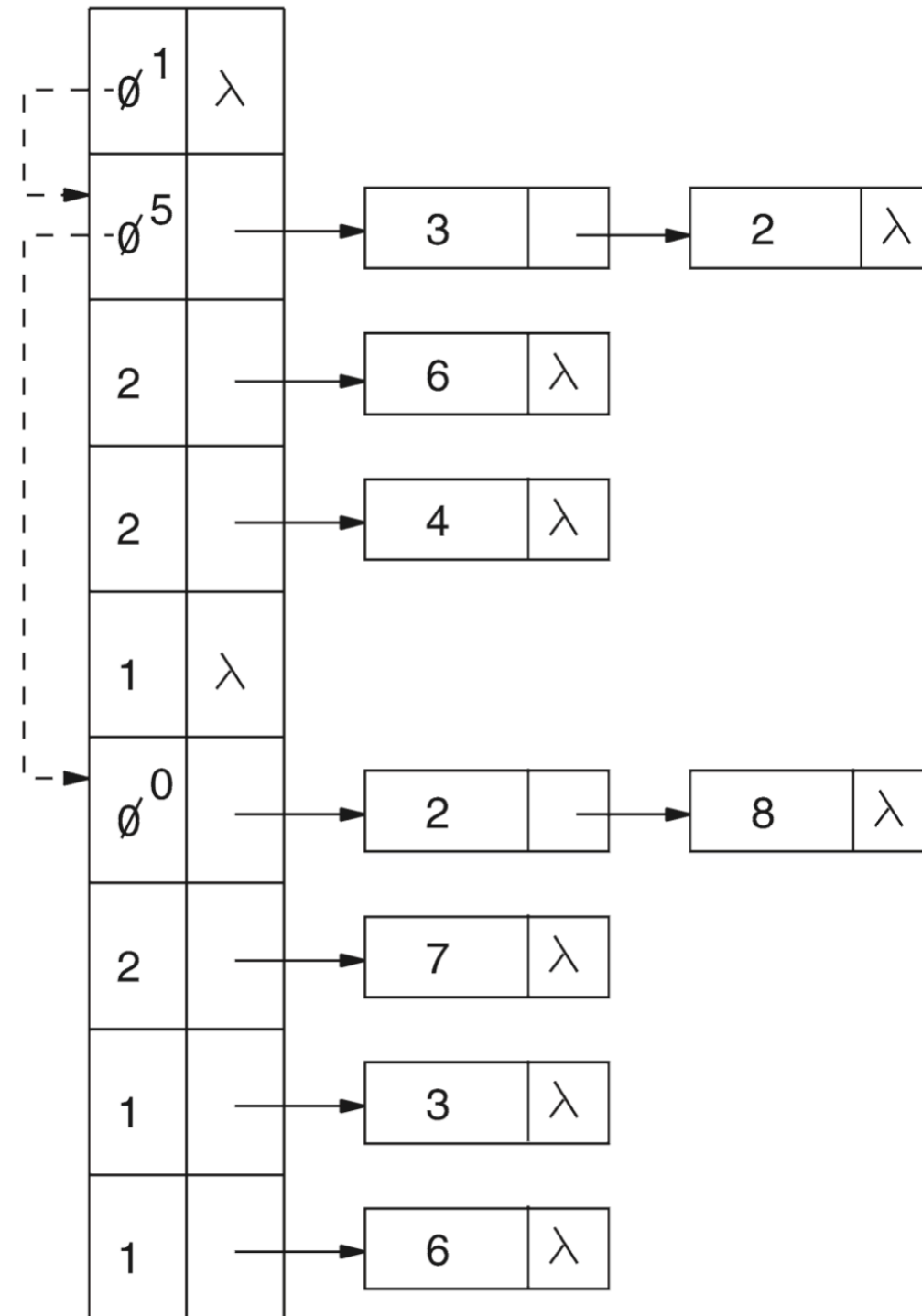


FIGURA 2.15 Armazenamento para a ordenação topológica.

Definições e main

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. /* Definicoes de tipos */
5. struct cel {
6.     int contador;
7.     struct cel *seg;
8. };
9.
10. typedef struct cel celula;
11.
12. /* Prototipos */
13. celula * inicialize(int n, int m);
14. void ordene_topologicamente(int n, int m);
15.
16. int main() {
17.     int n, m;
```

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. /* Definicoes de tipos */
5. struct cel {
6.     int contador;
7.     struct cel *seg;
8. };
9.
10. typedef struct cel celula;
11.
12. /* Prototipos */
13. celula * inicialize(int n, int m);
14. void ordene_topologicamente(int n, int m);
15.
16. int main() {
17.     int n, m;
18.
19.     // leia n
20.     printf("n:\n");
21.     scanf("%d", &n);
22.
23.     // leia m
24.     printf("m:\n");
25.     scanf("%d", &m);
26.
27.     ordene_topologicamente(n, m);
28.
29.     return 0;
30. }
```

Inicialize

```
1. celula * initialize(int n, int m) {
2.     int i;
3.     celula *cb;
4.
5.     // alogue um vetor de n celulas cabeca
6.     cb = (celula *) malloc (sizeof(celula) * (n + 1));
7.
8.     // inicialize as n listas
9.     for (i = 0; i <= n; i++) {
10.         cb[i].contador = 0;
11.         cb[i].seg = NULL;
12.     }
13.
14.     for(i = 1; i <= m; i++) {
15.         celula *p;
16.         int j, k;
17.
18.         // leia o par (j, k) – j precede k
19.         scanf("%d%d", &j, &k);
20.
21.         // crie uma celula e inicialize com o sucessor k
```

```

4.
5. // alogue um vetor de n celulas cabeca
6. cb = (celula *) malloc (sizeof(celula) * (n + 1));
7.
8. // inicialize as n listas
9. for (i = 0; i <= n; i++) {
10.     cb[i].contador = 0;
11.     cb[i].seg = NULL;
12. }
13.
14. for(i = 1; i <= m; i++) {
15.     celula *p;
16.     int j, k;
17.
18.     // leia o par (j, k) - j precede k
19.     scanf("%d%d", &j, &k);
20.
21.     // crie uma celula e inicialize com o sucessor k
22.     p = (celula *) malloc (sizeof(celula));
23.     p->contador = k;
24.
25.     // adicione na lista j, que contem os sucessores de j
26.     p->seg = cb[j].seg;
27.     cb[j].seg = p;
28.
29.     // incremente contador de k, que contabiliza o numero de vezes
    que k aparece como sucessor
30.     cb[k].contador += 1;
31. }
32. return cb;
33. }
34.

```


Ordene Topologicamente

```
1. void ordene_topologicamente(int n, int m) {
2.     celula *cb;
3.     int i, fim, objeto, indice;
4.
5.     cb = inicialize(n, m);
6.     fim = 0; cb[0].contador = 0;
7.
8.     // busque objetos sem predecessores
9.     for (i = 1; i <= n; i++) {
10.         if (cb[i].contador == 0) {
11.             cb[fim].contador = i;
12.             fim = i;
13.         }
14.     }
15.
16.     objeto = cb[0].contador;
17.     while (objeto != 0) {
18.         celula *p;
19.
20.         printf("%d -> ", objeto); // saida objeto
21.         p = cb[objeto].seg; // p recebe a lista de objeto
22.     }
```



```

11.         cb[fim].contador = 1;
12.         fim = i;
13.     }
14. }
15.
16. objeto = cb[0].contador;
17. while (objeto != 0) {
18.     celula *p;
19.
20.     printf("%d -> ", objeto); // saida objeto
21.     p = cb[objeto].seg; // p recebe a lista de objeto
22.
23.     while (p != NULL) {
24.         // armazene em indice o contador da celula (sucessor)
25.         indice = p->contador;
26.
27.         // decrémente de 1 a quantidade de vezes que indice atua como su
28.         cb[indice].contador = cb[indice].contador - 1;
29.
30.         // se agora indice atua 0 vezes como sucessor
31.         if (cb[indice].contador == 0) {
32.             cb[fim].contador = indice;
33.             fim = indice;
34.         }
35.         p = p->seg;
36.     }
37.     // objeto recebe o proximo objeto do encadeamento
38.     objeto = cb[objeto].contador;
39. }
40. printf("FIM \n");
41. }

```

Entrada.txt & Terminal

entrada.txt

```
8
9
1 3
1 2
2 6
3 4
5 2
5 8
6 7
7 3
8 6
```

Terminal

```
% g++ ordenacao_topologica.cpp
% ./a.out < entrada.txt
n:
m:
1 -> 5 -> 8 -> 2 -> 6 -> 7 -> 3 -> 4 -> FIM
```

Referências

- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian.
Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora:
LTC. 2010